Chapter 22. SRE in Diverse Environments

By Ana Oprea, Chris Jones, Derek Remund, Hannes Eder, Ian Schlesinger, Joanna Wijntjes, and Yuri Grinshteyn
with Carmela Quinito

The SRE approach pioneered at Google—which applies software engineering to production problems—has proven to be sufficiently effective to attract interest from organizations in many industries. We start this chapter by exploring a common situation: What if you're the first SRE at a small company, or considering hiring one? Where do you start? How do you improve or maintain service reliability without succumbing entirely to operational work and toil?

We then address organizations that struggle to replicate or adapt the SRE model due to technical, organizational, or process constraints. After outlining these common challenges, we suggest approaches to improve reliability practices under such conditions.

To make these scenarios more concrete, we discuss real-world examples of applying SRE in various environments: a rapidly scaling B2B startup, a multinational bank, a large-scale mortgage lender, and Google's autonomous driving company, Waymo. Finally, a section on security and privacy explores how SRE skills apply in a different context. While earlier sections detail how SREs communicate or divide tasks with separate organizational functions, this section explores a technical convergence where role boundaries naturally blur to defend the production environment.

SRE When Faced with Staffing and Organizational Constraints

When reading about SRE, it's easy to get the impression that an SRE team is always large. SRE books (including ours) commonly describe a follow-the-sun on-call model with engineers who are ready and available to respond to any outage, and incident response teams with a dedicated incident commander, communications lead, and operations lead, overseeing a team of engineers working to debug the problem.

This can leave you with the impression that an SRE is always accompanied by a large team of stalwart coworkers. Worse, it can leave you with the impression that you can't do SRE if you don't have a large team supporting you. This section outlines some of the challenges that small SRE teams face and offers practical advice on how to maximize your impact with fewer people.

The Solo SRE: Surviving and Scaling SRE in a Startup or as Part of a New, Small Team

Google SRE has been around long enough to see valued coworkers leave and start SRE teams at other companies. We've had the opportunity to share SRE practices with engineering teams outside Google, and in doing so we've learned how challenging it can be to get started without the many tools and frameworks that are available to us internally. But arriving as the first SRE or being on a small team at a company that wants to start doing SRE can be an exciting opportunity to focus on what's important and make a really measurable contribution.

The decision to hire the first SRE is often a reaction to a painful outage—or worse, a series of painful outages. The platform is fragile, release is manual and dependent on human oversight, finances are limited, and everyone is in firefighting mode. The outages may have become public enough that they are hurting the chances of the company being able to survive at all, and someone decides that having an SRE might help. Another common motivation might be that while no dramatic public outages are happening, all available resources are dedicated to keeping a highly fragile service just barely together so that from the outside it appears mostly reliable. In both scenarios, there is extremely little time available to write and deploy new features.

The reality is that when you are an organization's first or only SRE, you have several roles. You are the firefighter who steps in during an outage and leads the team back to a stable position. You are the builder who figures out what needs to be added to prevent the next outage from happening, or at least reduce its impact. And you are the evangelist who starts the process of getting everyone on board with a new way of operating. In a larger organization, you would be able to specialize—for example, in observability, storage, or building new tools. As a solo SRE, you own it all. Your stakeholders are the entire engineering team, the product team, and the founders themselves. You must balance restoring production stability (firefighting) with building the automation necessary to stop the fires in the future (engineering).

Table 22-1 compares different aspects of the role of a solo SRE with that of an SRE in a large company.

Table 22-1. Characteristics of the SRE role in different contexts

	Large company SRE	Solo SRE (startup)
Focus	Adding automation, reducing toil, building and reviewing designs of large-scale systems	Keeping the service running, empowering developers to be as self-sufficient as possible
Pillars	SLOs, error budgets	Basic observability and deployment safety
Time allocation	<50% toil, >50% engineering (note that 50% toil is not the target—the more toil gets removed, the more features can be written)	Aggressive prioritization, targets identified with the assistance of SLOs wherever possible; often starts at 80% toil

Next, we'll explore some guidelines for the lone SRE. They will also be valuable for the manager of that SRE, or someone contemplating whether to hire or move someone into their company's first SRE role.

Prioritization is key

As the lone or first SRE, it is important to acknowledge to yourself that you cannot fix everything at once. You must prioritize with great efficiency, focusing on the tasks that reduce the highest amount of toil or mitigate the largest amount of risk. Make your priority stack, share it around, consider recommendations, and do not add new material without removing other items. When you have a list of priorities:

1. Make your first targets as specific as possible.
2. Choose one of those targets, and say when you will have a design to address it (if a design is necessary) and when it can be cleared.
3. When you clear the target, update the list and show the improvement.

The sense of accomplishment you experience will encourage you to continue making progress by moving on to the next item on your list.

Here is a possible prioritization stack for the solo SRE, from most to least urgent (priority zero, or P0, to P3):

Stop the bleeding (P0)

Immediately stabilize the worst, most frequent failure modes. If the database crashes twice a week, fixing this issue is your primary task, even if it means putting off feature work. Absolutely everyone hates experiencing the same out-

age over and over. Getting this kind of repeating problem sorted will be a tremendous relief and help to build confidence in your work.

Establish visibility (P1)

You cannot fix what you cannot see. Implementing basic, end-to-end observability is the foundation of everything else in this stack. Don't let perfect be the enemy of good; use an out-of-the-box solution or what the company has already to get yourself on the road to having visibility into the services that you are responsible for. You'll be able to improve on this later, when you have more time or help. Concentrate on these areas:

Metrics

Instrument key business and system metrics (latency, error rate, throughput). Strive to move away from situations where your customers are reporting critical problems to you and toward finding the problems first.

Alerting

Focus on symptoms, not causes (e.g., "latency is high," not "CPU is at 90%"). Ensure your alerts are actionable and low-noise. Test new alerts, or at least enable them at the start of the business day, not when you head home for the night.

Logging

Make a place for the logs to go and a way for them to get there. This could be an Elastic cluster, or even a single machine when you're getting started. Logs should be written with a clear structure—JSON is nice because it's both machine- and human-readable. There are good reasons why engineers still love to add print statements when they're trying to debug a problem, and you want to make it easy for them.

Automate the scariest task (P2)

Identify the single process that has been causing your new coworkers the most fear, stress, or errors. This process is usually something in the deployment pipeline. Your first major automation project could be making deployment a one-click, rollback-capable, zero-downtime operation. Of these characteristics, being rollback-capable is usually the most important: this makes people more confident about making changes (because you can always roll back) and makes it less likely that people will try to fix forward—a route that can lead to outage upon outage. Automating this high-visibility task will further build confidence in your work.

Developer self-service (P3)

> Instead of fixing every developer-raised issue, invest time in creating tools and documentation that allow the development teams to handle routine tasks themselves. This is how you scale your impact without scaling your headcount.

Careful prioritization of the tasks in this stack will enable you to start tackling some of the deficits of being a one-person SRE team right from the start.

The first 90 days

Start with simple, centralized observability. You don't want or need a million dashboards—what you need are answers to a handful of questions about the service you are responsible for:

- Is it down? Check for pings and health checks.
- Is it slow? Check for latency metrics at the edge.
- Is it broken? Check for error rates or status codes.
- Is it driving users away? Check whether a bug or missing feature is frustrating them.

Use a lightweight integrated toolset (a single-vendor or open source solution is desirable here) that centralizes logs, metrics, and traces. Avoid putting yourself in the position of having to integrate multiple disparate systems—this kind of integration work is toil you can't afford at this juncture.

Leveraging AI for help

AI-powered tools can be a great assistant to someone attempting to jump-start a new SRE team. You can use these tools to automatically analyze the centralized logs, metrics, and traces that you are gathering.

By analyzing historical performance data from your metrics, AI tools can help identify baseline performance for key indicators such as latency and availability. This data-driven approach can provide realistic starting points for SLOs, reflecting actual service behavior and user experience; you can then refine these over time.

Natural language processing capabilities within AI tools can be applied to user feedback, support tickets, or error message patterns to gauge user frustration ("Is it driving users away?"), offering insights beyond traditional metrics. The goal is to leverage AI to surface actionable answers from your observability data, reducing the need for extensive manual integration, dashboard proliferation, and guesswork in SLO definition.

Playbooks

Playbooks can be surprisingly controversial, and for some people, the idea of taking time to work on them when there are so many fires to fight seems like a bad use of time. However, well-written playbooks can serve multiple purposes. They help people fix problems when you are away, facilitate new hire onboarding, and can function as pseudocode for future automation—you write down, test, and fix up your playbook, and it becomes version one of the automation needed to address the problem going forward. It can be frustrating to have an outage where you know what mitigation is required, but no one knows how to perform it because the one person who does is on vacation, or is in the hospital, or has left the company. If you have put in the time to ensure that there is a straightforward way to roll your service back to a known good version, you also need to put in the time to ensure that other people know how to do the actual work—possibly at 2 a.m., when they are not at their most alert.

Start by creating playbooks for the five most recent outages (e.g., "database reads/writes are too slow," "API is serving 500s," or "failed deploy"). Write these playbooks for the non-SRE, because if you are on a small team, it may very well not be an SRE trying to follow the steps. Make the playbook useful to people in different roles who want to help recover the service—say, a developer who has discovered that they introduced a problem, or an operations person who was paged. This playbook is your opportunity to empower developers and other skilled personnel.

One of the first wins you can get from a nicely written playbook is the ability to delegate initial troubleshooting and diagnosis to the larger team: the playbook walks someone from that team through initial checks about the problem. It should also inspire you to create a good usable toolset—if you have a tiny bash script that works on any and all problems you are debugging, productionize the key elements of that script so that other people can share in the work. The team may still need to escalate to you, but when they do, they will have already completed initial troubleshooting, saving you time and effort. Once the outage is resolved, you can make updates that will get the team closer to being able to resolve the problem without needing to escalate to you at all.

As you learn more about how to operate the service, and as changes are made to the service, playbooks can become outdated. Establish a culture of making small, fast improvements as they are observed, and be sure to solicit feedback on what needs to be made clear. For example, a team might be experiencing a rollout that gets into a failure mode that, while not service-impacting, is frustrating. In this situation, you can create a playbook showing on-callers how to fix the problem, and then iterate and make improvements as different people encounter the problem.

If you successfully engage the team in creating playbooks, you can leverage an LLM to make them findable and interactive. Retrieval-augmented generation lets you index your playbook library so that when an incident occurs, you don't have to sift through documents manually; you can ask what the steps are to roll back a specific deployment, effectively transforming your static playbook into a queryable knowledge database and speeding up your incident response.

The playbooks should become important resources during outages. You'll want to identify a low-overhead way to ensure the most important information is available even when the service is completely unreachable. There are a number of ways to handle that (GitHub repo, panic package on a local machine, offline compilations). Identify what makes the most sense for your situation and target the most critical information so that it is available when you need it.

Scaling yourself

You are just one person. You don't have two times six people in an on-call rotation nicely arranged across two time zones.[1] You may be the only SRE at a company preparing for an important launch. As discussed previously, it is likely that you were hired after a bad outage, so it's a high-pressure situation and everyone around you has expectations. In these conditions, it's going to be really important for you to scale yourself.

Your initial SLOs and SLIs need to be focused on the most critical user-facing parts of the service. While the core implementation is going to be like that at a much larger company, it will distill down the results into something very focused. To address the priority items outlined in "The first 90 days", identify the critical user journeys and set up a prober service to measure important SLIs like availability, latency, throughput, and errors.

Once your SLOs are accurately defined, you can use them as a reference to determine whether a filed bug requires immediate action or if the SLO needs adjustment. These checks will provide a foundation that you can refine and improve on and share with your team and partners.

At a company of any size, it's important to build a culture of cooperation with the developers. However, this is especially important when you are the single SRE, or an SRE in a small team without round-the-clock coverage, as developers are best positioned to help you improve the reliability of the service. SREs have a reputation for saying "no" a lot, but they also have a reputation for showing up when a confusing and upsetting outage is in progress and doing the work required to create order, apply mitigation, and (eventually) build understanding. Make yourself someone who enables rather than blocks other

people, and the team and culture will thrive. Be on the lookout for cleaner, easier solutions and for complexity that might cause problems as the feature evolves. Writing and supporting code at a startup can be a big trade-off game: do we build this time-consuming, robust solution, or do we choose this simpler but much faster solution? Both of these options have value, and if you create a culture of cooperation with the product development team, you can balance the options together and make better choices.

You can establish an on-call rotation composed of the development team and yourself. You have the additional goal of collecting information about what fails and making improvements to observability and SLOs, but the product development team is most likely to know that the problem happening today is the result of a bug in the code they wrote the day before. Being part of a shared on-call rotation is a great opportunity to evangelize SRE norms. If there is an outage, you can take on the role of incident manager and demonstrate a calm, effective way to organize the debugging and mitigation. After the outage, you can determine the causal factors, publish the postmortem internally, and get feedback on proposed action items.

An important part of collaboration can be reviewing each other's work. We recommend implementing a code review process even if people do some (or a lot of) initial complaining. A culture of code review is self-reinforcing, getting people accustomed to the idea that having an extra set of eyes on a change is not a drag on velocity but an additional way to spot problems. It's helpful to point out here that reliability actually *improves* velocity, because developers have fewer outages that demand their time. Buggy code submitted fast that needs to be rolled back is an even greater drag.

A review process can be the start of growing an SRE culture, and in the case of the solo SRE, it can be the beginning of growing your team. A review provides the following benefits:

- At least one other person knows about the change that was just introduced, even if they miss the bug that just got written. They've signed off on the change, so they share in the responsibility for what happens next.
- Junior engineers learn from reviewing code written by senior engineers, and senior engineers reviewing code written by juniors can provide coaching.
- A reviewer can provide helpful suggestions, such as adding unit tests to the code.
- A reviewer can catch problems, even if it's only one problem in five pull requests.

Many tools and automated checks can identify problems with a pull request before it is submitted. Increasingly, AI can also be leveraged to check for both common and uncommon failures. As the first SRE, or the person starting the SRE team, an important job will

be to establish code review norms and get the developers out of a mode where they submit whatever they want, whenever they want, without anyone else looking at it. Instead, SREs and developers should write up change requests with clear descriptions and send them to a peer for review.

You can use AI to help vet changes; however, at this stage of your development, make it a goal that all changes are reviewed by at least one other human. This will help catch errors and move you toward a culture of accountability. As you mature your service operations by defining SLOs, integrating them into monitoring, and building trust in consistently meeting them, you can judiciously reduce human oversight, confident that any service degradations will be rapidly detected and mitigated.

SRE Applied in Hybrid and Legacy Environments

In the software industry, "hybrid and legacy" typically refers to siloed environments with a mix of internally developed software and acquired systems or components. As other enterprises—especially those operating in legacy or hybrid environments—attempt to adopt SRE principles and practices, they typically face challenges in two major areas. The first is the constraints imposed by organizational structures, processes, access models, and related requirements. The second is technical and environmental limitations. This section provides suggestions for how to tackle these challenges in your own environment and context.

Organizational, Process, and Access Constraints

Bridging the gap between security requirements and production stability requires thoughtfully constructed operational boundaries. The tension between these priorities becomes most acute during incidents, when rapid mitigation must be balanced with explicit authorization and auditability.

Access and authority during incidents

To accurately and effectively detect, triage, and mitigate a production issue, an engineer responding to the issue requires the following:

- Sufficient visibility into the service, its dependencies, and the underlying infrastructure to understand what's going on
- Sufficient access to make the changes to these systems that are needed to mitigate an ongoing incident

Google SRE on-callers have the access they need to understand the state of the system itself via metrics, logs, traces, and other debugging information without having to ask for permissions or escalate to another team. As a direct result of a years-long technical and cultural investment in a common production platform and set of tools, they can get similar information for dependencies from the same centralized observability systems without needing to ask other teams to provide it. They can then take the actions necessary to restore their own service to a known good state, such as rolling back a bad configuration or code change, or redirecting traffic around physical network or infrastructure issues. They can even take actions for services they don't own directly by temporarily escalating their own privileges or (more commonly) by preparing a rollback of a bad change and sending it to the service owners. If they make a mistake at any point, they are not personally held responsible for making the outage worse, and therefore they feel safe doing what they deem to be best in the moment.

We have seen a number of common challenges arising from organizational, legacy, or even regulatory constraints. Incident response personnel in a large network operations center (NOC), for example, may only have permission to inspect the production system at a surface level and may not be able to make changes themselves. The NOC may also be staffed through an outsourcing arrangement, where incident responders are not employees, operate with limited access, and must escalate to in-house teams to gain additional access or make changes.

In these contexts, SRE principles can improve incident response by formalizing the transition from steady-state operations to active crisis management. One effective approach is to designate an incident commander (IC) with the explicit authority to direct the response, regardless of the organization's size or existing hierarchy. While the IC may not possess "all-access" technical permissions (the "superpower" to touch every system), they hold the *mandate* to coordinate those who do. For smaller organizations, this may simply mean the IC has the authority to reprioritize the work of any available engineer. To bridge the gap between authority and physical access, define predefined emergency access paths, such as time-bound privilege escalations, that the IC can authorize for responders. Incident commanders bypass organizational silos because the organization grants them full authority for the duration of the incident.

While granting broad system access to engineers accelerates incident resolution, it undeniably introduces a significant attack surface and potential trust problems. To mitigate risks of malice, incompetence, or external coercion, such autonomy must be paired with robust technical guardrails. Organizations can minimize the blast radius of any single individual's mistakes or malicious actions by implementing least privilege access by default, requiring multiparty authorization (e.g., a two-person rule) for sensitive data ac-

cess, and maintaining immutable audit logs of all production actions. Create systems that require that every high-level privilege escalation be short-lived, logged, and justified, and use them to empower on-callers to act decisively during outages, while also monitoring and auditing their access to sensitive user data such as chat logs or financial records. At Google, such systems govern, audit, and control all access to production systems, whether during incident response, routine troubleshooting, or other operational activities.

Autonomy and authority to prevent incidents

To improve the reliability and supportability of systems over time, engineers who respond to production issues require the following:

- Engineering capacity (time) allocated to postmortems and identifying improvement opportunities. This allocation depends on leadership prioritization.
- Commitment from the teams that own, build, and maintain the service to invest engineering capacity in implementing the resulting action items.

In the ideal case, SREs work in partnership with the service owners or developers. They work jointly with product engineers to include reliability considerations in designs and address reliability issues that are uncovered in production. At Google, when this relationship breaks down, SREs are empowered to walk away from the service if they cannot support it while adhering to operational work limits (see Chapter 7).

The ability to walk away may not be possible in your case. In fact, it may be that the engineers charged with production responsibilities are expected to support the service and defend its reliability no matter what happens on the other side of the proverbial fence. One way to improve this scenario can be to run a toil audit and use its results to negotiate for time to invest in additional automation. The objective is to quantify the most frustrating and frequent manual tasks. The staff who perform the work can identify and list these tasks, quantify the time savings and other benefits (such as reliability improvements) that might be realized by automating them, and leverage this data with management to budget time for automation work. This approach reframes the issue from a suggestion for change ("we should automate") or a complaint ("we're doing too much toil") to a business justification ("automating X repetitive task gives us Y hours back to work on product features").

In some cases, automation may unlock entirely new use cases that weren't possible before. Automatically scaling infrastructure to meet user demand is a simple example of what's possible. Another example is setting up probers to validate application perfor-

mance and availability—especially while the application is undergoing disaster recovery, chaos testing, or failover testing. Many users perform simple manual validation during a disaster recovery test to confirm that it "worked." Having reliable SLIs derived from prober traffic against test environments that track application health before, during, and after the test provides a much more complete picture of whether the experiment passed or failed.

While automation is a core pillar of SRE, it is not a "set and forget" solution that yields immediate time dividends. As noted by Lisanne Bainbridge in her work on the "Ironies of Automation" (see the sidebar "The Ironies of Automation"), shifting tasks from humans to machines often replaces simple manual work with more complex supervisory and maintenance tasks. Every automated system introduces a new surface area for debugging, integration challenges, and the eventual need for rewrites as the underlying environment evolves. Therefore, the goal of automation isn't just to eliminate toil but to intentionally trade high-frequency manual labor for the lower-frequency, higher-value work of maintaining the automation itself.

Another approach is to integrate incident responders into the postmortem and action item process. If the support team's insights fail to translate into system improvements, it is usually because their experiences are not being formalized or recorded as organizational knowledge. This can also occur when the primary responders are disconnected from the product development team and its knowledge. The SRE model uses postmortems as the formal mechanism for closing these gaps and driving systematic change. (See Chapter 10 for an in-depth discussion of blameless postmortems.)

This focus on learning from incidents in a blameless way gives a formal voice to the responders' operational reality: their inability to fix a problem or suggest a better playbook becomes an *organizational* failure to be addressed through action items, ensuring the problem is surfaced to leadership and assigned an owner for resolution. Finally, in regulated enterprises accountable to government authorities (such as the US Federal Aviation Administration or Food and Drug Administration), when things go wrong, the postmortem might be used as formal documentation demonstrating that corrective and preventive action (CAPA) is being taken. The CAPA itself then becomes a documented regulatory requirement, effectively bypassing routine bureaucracy and elevating the priority of the reliability fix. If you work in such an environment, you're likely well familiar with these regulations; if not, your local compliance office should be able to help.

Technical Constraints and Obstacles

In cases where an organization builds all the software it runs, the SRE function has the freedom to write additional software to accomplish its objectives. SREs can also directly modify the code of the services they support to improve scalability, reliability, and efficiency. However, in practice, most enterprises have significant software investments in third-party commercial off-the-shelf (COTS) software, whose code cannot be easily modified, if it can be modified at all. But even when operating in such an environment, there are opportunities to employ SRE practices and principles for improvement.

Most enterprises do not build their software stack in-house and instead rely heavily on third-party COTS software. This is true not just for business applications but also for the infrastructure and tooling used to run the environment, such as monitoring, provisioning, and CI/CD pipelines.

Regardless of the software's origin, the SRE function provides the most value when empowered to develop its own automation and software layers. Even when working with closed systems that cannot be directly modified, SRE teams can identify high-leverage opportunities to build custom wrappers. These tools allow SREs to extend existing capabilities, isolate proprietary systems, and inject the standardized reliability practices necessary to maintain a stable production environment.

Closed systems

When you cannot change the internal workings of a third-party application, focus on stabilizing the systems and processes that surround that application. Work with what you have, rather than trying to change systems that are “nearly impossible” to run the SRE way. While the core proprietary software cannot be managed via code, the deployment, networking, and operational components that support it can be. Infrastructure-as-code (IaC) tools, such as Terraform, can provide improved consistency, speed of deployment, and disaster recovery for system infrastructure components. These tools should help you reduce risk without needing access to the source code of the proprietary software itself.

Running third-party systems in production often requires significant operational overhead and manual repetition (i.e., toil). As recommended previously, audit the toil and identify the most frequent, repetitive manual tasks related to supporting the system. Create scripts to handle common mitigations (e.g., restarting failed services or clearing caches after an incident), instead of attempting complex, self-healing automation. Scripts can also help with post-incident cleanup and validation, reducing operational debt and making the system easier to support.

Beyond managing the infrastructure, SRE can develop a robust internal API or wrapper layer—often referred to as an anticorruption layer. By building this intermediary layer that the organization fully owns, SREs can programmatically control the communication path to the third-party software. This allows the team to inject critical reliability practices that the COTS product may lack out of the box, such as standardized observability, granular rate limiting, and validated configuration management. Rather than simply reacting to the behavior of an opaque system, this architectural approach empowers SREs to proactively govern how the system interacts with the rest of the environment, effectively turning a rigid third-party tool into a more predictable and manageable component.

Heterogeneity and silos

Google's current standardization on Borg-based infrastructure (see "Google's Production Environment") and associated tooling enables the SRE organization to operate many (but by no means all) services in a uniform way and apply generic mitigations.[2] But what happens when you're operating a complex, long-standing, and diverse technical environment, or one that is siloed with multiple distinct technical verticals? How do you apply modern SRE practices in this context?

Pragmatism is key. When faced with these constraints, SRE teams should prioritize practical, customized solutions over rigid adherence to a standard framework. To maintain momentum, consider the following approaches:

Focus on generic mitigations

Identify ways to mitigate incidents that can be applied quickly and reliably and do not necessarily require deep system introspection.

Adopt principles incrementally

Instead of a wholesale transformation, introduce SRE concepts in stages (see "Practical Considerations").

Focus on limited scope

Start within a single vertical or system, tailoring the implementation to "work with what you have" rather than waiting for infrastructure upgrades.

Define context-specific goals

Set reliability targets that make sense for the individual system's operational reality rather than pursuing a uniform standard across the entire enterprise.

Prioritize preparedness for rigid systems

> For legacy or third-party systems that are difficult to modify, shift focus toward disaster testing, certifications, and live drills to handle emergencies caused by external factors. Treat highly reliable and redundant systems similarly.

For a real-world example of these practices, see "Case Study: Large Multinational Investment and Consumer Bank".

Next, introduce standardization through the systems that manage and operate the bits you cannot change, rather than the bits themselves. Build a lightweight, standardized toolchain and processes that sit *above* the disparate infrastructure, for example, using IaC and uniform monitoring tools. Tools like Terraform can improve the consistency and speed of deployments and make disaster recovery easier by managing the provisioning and configuration of all new and modernizable infrastructure components. Because this is a complex, time-consuming process involving significant organizational back-and-forth, standardization should be approached incrementally, treating it as a long-term evolution rather than an overnight transformation.

Standardization ensures new systems are deployed uniformly and reduces future complexity. To unify monitoring across diverse technical stacks, introduce symptom-based alerting (Chapter 8). Since the heterogeneous systems may have widely varied internal performance metrics, focusing alerts on direct indicators of user happiness (e.g., latency or availability) standardizes the incident response experience. This approach moves the focus from the complex internal details to the measurable impact on the business.

Humans in the loop

In some cases, manual procedures are preferred or required, or human verification has to precede automated actions. Following the model used in the aviation industry, implement automation selectively, ensuring that core system implementations remain subject to manual inspection and verification. This preserves the benefits from the speed and repeatability of automation, which produces higher-quality standards, and retains the trust of the human decision chain. Consider using automated remediation actions for predefined breaches of SLOs or specific, high-urgency conditions where the risk introduced by fully automated response is lower than the risk inherent in slower human intervention. As a specific example, adding capacity when detecting a substantial increase in inbound traffic could be a good candidate for automated response as a well-understood, acceptable risk.

If automation fails or leads to an incident, use the blameless postmortem process to analyze the systematic failure (e.g., a flaw in the automation tool or configuration) rather than an individual mistake. This process should help build the psychological safety required for engineers to continue proposing and building automation without fear of career repercussions, as well as improving the automations themselves over time.

In regulated industries such as aviation, finance, and healthcare, trust automation only when verified by a human—failover may be automatic, but core system implementations often require manual inspection. In such environments, you can focus on highly specific, low-risk, high-return automation efforts, while formalizing the human role in the process.

Ultimately, even though applying SRE principles to hybrid and legacy environments presents unique technical constraints—including reliance on third-party and legacy infrastructure and dealing with heterogeneous and siloed systems—these challenges can be effectively addressed. The key is to focus your efforts on the surrounding systems and processes, embrace pragmatic and incremental solutions tailored to specific contexts, and implement highly targeted automation for toil reduction—all while leveraging blameless postmortems for continuous improvement.

Implementing SRE in hybrid and legacy environments requires a pragmatic balance between ideal reliability practices and the reality of ingrained organizational and technical constraints. While technical debt and siloed access often hinder the speed of incident response, the path forward lies in fostering a culture of accountability and incremental standardization. Empower engineers with greater visibility and the authority to act during outages to begin to dismantle the barriers that lead to high mean time to resolution (MTTR). Seek to adapt core SRE principles—like a blameless culture and a focus on automation to fit your needs—rather than trying to perfectly mimic a cloud native model or "be more like Google." Aim to transform reliability from a specialized task into a shared responsibility and to enable even the most fragmented systems to achieve modern performance standards. Through persistent advocacy and technical evolution, you can move from mere maintenance toward a sustainable, proactive operational future.

Real-World Examples

Even in cases where the SRE adoption effort is well resourced, building a practice in an existing enterprise can come with its own set of challenges. Over the last decade, Google engineers have worked with a variety of customers to assist them in standing up SRE teams and instilling SRE practices, helping them build and run reliable, scalable sys-

tems in the cloud. These customers have included large enterprises, well-funded startups, and even public institutions. The case studies that follow describe some of these experiences and illustrate real-world approaches to retrofitting SRE into an existing organizational structure.

Case Study: Rapidly Scaling B2B SaaS Startup

A recently funded SaaS startup needed to stabilize its platform after experiencing sudden growth that pushed its ad hoc engineering practices to the breaking point.

The scenario

Founded two years earlier, the company experienced a massive spike in user traffic once it found product-market fit. The initial two-person engineering team had grown to about 20 developers in response to product demands, but the operational model remained stuck in the "garage" phase: everyone pitches in, everyone wears all hats, everyone focuses on the most recent or prominent problem.

On top of the cultural issues, deployments were more manual and less well documented than ideal, and outages were frequently discovered through user feedback surfaced to the founders. The core engineering team was thus trapped in a constant state of firefighting, making velocity unpredictable. Leadership identified these issues and brought in a new hire dedicated to "bringing order to the chaos" through SRE.

The adoption approach

The new SRE leader faced operational debt and overloaded engineers. Two engineers, one senior and one junior, were tapped to form the core of a new SRE group, bringing their number to three. They sat directly within the engineering group as fully fledged software engineers, but with a focus on the SRE mandate. The SRE group identified four critical priorities to tackle initially:

Establish visibility

They performed an immediate review of observability tooling and established standards for metrics, logs, and traces, which they partnered with the rest of engineering to systematically implement.

Automate the biggest source of toil

Issues with deployments caused the majority of outages (at least anecdotally, though this was later confirmed by updated data). The SREs focused on cleaning

up the deployment automation and introduced "one-click" rollbacks for most rollouts.

Create playbooks

They gathered tacit knowledge from the team and wrote playbooks for the top 10 most common failure modes.

Implement basic SLOs

With the updated monitoring in place, they worked with the product manager and the founders to set initial SLOs that reflected user experience expectations. These were imperfect, but they formed the basis for reevaluating reliability from the users' perspective.

The SRE group also established a formal on-call rotation that included all of the engineering team.

The results

By focusing on high-impact automation and developer enablement, rather than simply owning on-call and fixing the bug backlog, this initial group of SREs successfully transitioned the company out of the "constant firefighting" phase. Within six months, the following improvements were solidified:

Deployment confidence

Rollouts shifted from high-stress events to routine, frequent operations. Not only did mean time to failure (MTTF) increase, but developers took more risks (which paid off) due to higher confidence in rollbacks.

Reduced MTTR/MTTD

As use of SLOs proliferated, symptom-based alerting led to reduced mean time to detection (MTTD), while MTTM/MTTR improved via playbook usage and more consistent, less chaotic triage. The team saw an approximate 60% reduction in overall downtime across its identified critical user journeys.

Culture of collaboration

The inclusion of the full development team in the on-call rotation, and the developers' enablement by the SREs, ensured that a "throw it over the wall" mentality wouldn't develop with the growth of the SRE team.

Data-driven prioritization

> SLOs and a more detailed reliability backlog became primary inputs into the team's prioritization, allowing for more objective decision making and ultimately increasing product velocity. One major feature was shipped a full 60 days earlier than originally planned.

The SRE group has since grown and now operates as a separate team alongside the development team, reporting through the same engineering leadership chain. With product velocity unlocked, the development team has grown as well, and the engineering group has stabilized at roughly a 7:1 developer-to-SRE ratio. Approximately half of SREs are direct hires, while the other half come from the development team. Although SRE ownership of on-call has increased over time, all developers still participate in some on-call rotations. Operational work for developers now accounts for about 5% of their time, down from nearly half when the first SREs joined the team.

Case Study: Large Multinational Investment and Consumer Bank

A key financial institution was seeking to redefine the customer experience for critical products like trades and wire transfers, and looked to SRE to make it possible.

The scenario

A storied company with a long technological history, over time the bank had adopted a large variety of infrastructure environments and overlapping tools, each used within one or more silos as a point solution to a particular problem. Many of these choices were driven by a combination of regulatory requirements and the need to interoperate with external organizations, as is the nature of a key player in the global financial system. Similarly, the bank had developed a complex web of internal and outsourced capabilities over the years, so even basic functions such as program management were performed differently by different teams, depending on the business unit.

Even the most venerable institutions can recognize a need to improve. After a series of outages across the industry, introspection led the firm to a stark conclusion: it knew where it had problems, and when, but its ability to react to and resolve complex outages was heavily limited by a siloed organization and a culture of blame. These obstacles made navigating the complex environment incredibly difficult.

When Google first began working on SRE transformation with this organization, a leader laid out a core issue: the company was composed of several distinct silos, each of which

had existed "since before employees had computers on their desks" and acted as a quasi-independent entity, with leaders that expected near-total control and decision-making authority. Any method of building an SRE culture would have to avoid appearing to infringe on the sovereignty of these verticals.

The bank did have some key strengths. Average employee tenure was quite high, which meant domain knowledge was strong. By market necessity, the firm also had a relatively high degree of technical sophistication in some areas. As a financial institution, clear ownership of success and comfort with discussing risk were cultural values that could be leveraged to build an SRE model.

The adoption approach

A simplistic, universal rollout of SRE practices such as SLOs and role-based incident response was likely to get bogged down in divisive debates. The diversity and sovereignty issues required a capabilities-based approach. We focused on identifying and pursuing specific SRE capabilities within individual verticals to address their unique needs. Standardizing the implementation of those capabilities would follow. This approach could also address the variety of infrastructure environments by building a standard capabilities "surface" that abstracted away their differences. Some of these capabilities could be required as a baseline, while adoption targets could be customized for each vertical, or in some cases even more granularly, allowing the different businesses to maintain a sense of control and ownership.

In practice, the approach proceeded like this:

1. Work with a vertical to identify existing capabilities and key pain points.
2. Identify reliability targets for key services.
3. Align the combination of pain points and requirements for each vertical to a capabilities map,[3] classifying capabilities as "preexisting," "must have," "nice to have," "unneeded," and "explicitly undesirable."
4. Identify commonalities between the maps, some of which will exist because of our gentle prodding as coordinators.
5. Establish a subset of the commonalities as baseline requirements for production applications, and have it ratified by the verticals.
6. Work with each vertical to define an order and a timeline for gaining their needed capabilities.

Over the course of two years, SRE teams across several verticals established capabilities such as centralized SLO tracking, role-based incident response, and automated rollback.

While they didn't always act in perfect lockstep, this semicoordinated approach kept objections to a minimum while guiding the process. In the end, they had a consistent approach to SRE in which each vertical retained ownership of its reliability operations while contributing its improvements back to the whole.

Two other key strategies, one organizational and one technical, contributed to this success. Organizationally, it was crucial to designate a senior executive to advocate for reliability and the SRE transformation at the highest levels, to maintain momentum over the course of a multiyear change effort. Ideally, the person who takes on this "chief reliability officer" role is someone at or near the C-level who is able to act as a lighthouse, providing consistency, inspiration, and thought leadership as inevitable challenges arise. In this case, given the size of the organization, the role was filled by a managing director. This individual successfully leveraged their seat at the table to present a reliability "meta vertical" of sorts. This was a smart approach, considering the bank's preexisting organizational patterns: executive leadership understood the concept of a vertical and thus were primed to take in feedback from the owner of this critical reliability effort.

Technically, to deal with the vast landscape of overlapping tools and to minimize early conflicts over ownership, we focused on adopting "meta tools" that could plug into teams' existing software and processes to the greatest extent possible. These meta tools also helped speed up the process of establishing SRE capabilities. A secondary effort to reduce the tool space would then happen at a later stage. For example, we used dedicated SLO measurement and reporting software (a third-party SaaS product) that could integrate with all the major incumbent metrics and logging systems, so that the success of building initial SLOs didn't hinge on exactly how individual service teams had built their existing monitoring.

The adoption of similar SRE practices across verticals did its part to reduce blameful behaviors, but some issues remained, particularly with regard to severe multiservice incidents. This area was targeted directly by building a globally standardized incident response process and then practicing it, often and in depth, through disaster role-playing exercises. During these exercises, we put team members in a variety of roles, including ones outside their typical teams. A standing postmortem review committee, with representation from all SRE-aligned verticals, was established to guard against regressions into assigning blame and to ensure the completeness and actionability of incident records and follow-up action items.

The results

The work continues, as this bank is a very large organization, but after nearly two years of efforts, many teams have transformed the way they do business. Some of the largest changes include:

- SLOs that are tracked and reported the same way across verticals, allowing for better coordination between service teams and more confident distribution of resources based on need. MTTR was reduced overall, in part due to increased efficiency in mustering incident response teams.
- Incident response processes that are consistent while still allowing local service teams to have a major hand in resolution, rather than a centralized team dominating incident response and slowing MTTM/MTTR. This contributed to the lower MTTR by reducing escalation steps and decreasing time to console when human intervention is required.
- Fewer layers of service management, requiring less overhead for coordination and unlocking greater velocity per individual. In this case, the total number of personnel didn't change significantly, but their distribution shifted toward wider, more flexible layers rather than a deep, five-plus-level escalation stack.
- Creation of aspirational service level targets that are seen as valid and useful across verticals, in contrast to the siloed mentality that prevailed before the shift. This led to a flywheel effect of SLO proliferation across the wider business.
- Reduction of postmortem outputs focused on assigning blame and measurable reduction in repeat incident frequency.

Case Study: Large-Scale Mortgage Lender

A top mortgage provider was looking to stand up an SRE practice to help it meet the closing timelines borrowers expect.

The scenario

This lender had chosen to invest heavily in technology, as its market strategy involved working with independent brokers who brought in borrowers primarily based on the ease and consistency of the mortgage experience. It had also decided to grow these capabilities largely in-house, with an engineering staff of thousands building and running the software stack at scale.

In its hurry to expand its market, the firm had focused far more on "building" than "running," with one or two designated individuals acting as de facto on-callers within each

software team, or in some cases across multiple teams. While leaders at the top did champion the user experience, there was a disconnect between their measurement of best-case scenarios and the reality for users impacted by outages or performance issues. Reliability might temporarily become a priority following an outage, but the subsequent improvements would be unevenly distributed across teams and often short-lived, as no one really owned production standards and the individual teams' efforts were uncoordinated.

A few key individuals in the company eventually recognized these efforts as positive but unsustainable, and they sought to build an SRE culture and organization to allow the company to truly practice user-centric engineering.

The adoption approach

The company began its journey toward SRE by identifying key developers, DevOps engineers, and de facto SREs within a few of its most critical service teams and putting them through an SRE development workshop over the course of about three months. The participants discussed SRE cultural practices, learned and practiced the art of building SLOs, and defined aspirational standards for incident response, postmortems, on-call planning, toil management, and similar functions. The goal was to plant the seed of SRE in their flagship services.

The next priority was to define an SRE engagement model. Because of the size of the overall organization and the need to prove the concept before mass investment, leaders chose to start with a small centralized SRE team that would engage with service teams in the following ways:[4]

- Provide advice, guidance, and assistance as needed through an issue flow.
- Provide time-limited, 90-day engagements with teams to solve specific reliability problems or perform onboarding into the SRE approach.
- For the most critical services, provide one to three designated SREs aligned with those teams and dedicate a certain portion of their time (50–75%) to making reliability improvements for those services.
- Build ongoing reliability capabilities for rollout to all service teams.
- Provide training for new service-level SREs through a three- to six-month rotation on the centralized team.

Initially, this team was staffed mostly by individuals who had participated in the SRE development workshop. Once it had demonstrated value through a few key engagements with product teams, it was time to begin expanding.

The centralized team developed an eight-week, eight-hour-per-week SRE enablement program to prepare service teams for SRE adoption. Participants learned SRE and SLO fundamentals and were exposed to the tooling and standards established by the centralized team. They then took those lessons back to their teams and, where appropriate, engaged further with the centralized team through one or more of the options in the aforementioned engagement model. This program was run multiple times to expand the reach of the nascent SRE practice.

The results

The centralized SRE team now sits within a platform and reliability engineering vertical and works with the center of excellence team to develop production standards. More than six dozen practitioners have become SREs or embedded reliability-focused developers within their teams, and they've built a slate of SLOs that can now be used to better coordinate incident response and deal with the unsustainable operational load. It is still early in this transformation, but SLO attainment is already trending in the right direction, and teams report a reduction in contradictory management directives.

One unforeseen improvement has been a reduction in "weaponized blamelessness," where the desire not to point fingers leads to an increase in MTTM/MTTR as teams focus on their own responsibilities and use blamelessness defensively to dodge ownership. The spread of clearer SRE culture standards, and the embedding of SRE champions across critical service teams, continues to move the culture toward a truer approach to blamelessness, where the teams can focus on technical and operational realities without fear of political fallout.

SRE Managing Real-World Safety: An Example with Waymo

The Incident Management at Google (IMAG) protocol (discussed in Chapter 10) is fundamental to maintaining consistent reliability for an organization. A core feature of IMAG is establishing an incident response team with clear roles at the outset of an event. These practices did not originate in a vacuum; as Google's structured approach to incident response quickly evolved beyond the basic "everyone pitches in and helps" model, it drew on a myriad of existing paradigms to develop a unique and effective incident response culture.

The first edition of this book included a chapter dedicated to lessons learned from other industries, which examined high-stakes fields such as nuclear power, aviation, telecommunications, and medicine—environments where failures have immediate real-

world safety consequences. The emergencies these industries face can broadly be termed *crises*, and responding to them *crisis management*.

Most SREs in the tech industry today are focused on software uptime in a more product-oriented sense and are less directly concerned with safety-critical environments. Software incidents are typically described as outages. Waymo, on the other hand, must balance those same product expectations with additional physical safety standards. This difference is reflected in Waymo's incident response practices, which we'll describe in detail here.

SREs at Google and throughout the tech industry hone IMAG skills as a core responsibility; debugging and resolving major outages is a hallmark of the profession. When SREs are well versed in incident management, these skills can often be applied with surprising ease to nontechnical crises, enabling them to provide effective incident management for company-wide incidents that extend beyond technical issues. In doing so, they can deliver outsized impacts, drawing on the cross-disciplinary roots of IMAG that originated outside Silicon Valley. The following sections examine what this ability means in practice and when an organization should consider leveraging SREs beyond their traditional technical incident response role—as the spearhead of company-wide incidents—through a case study of Waymo SRE's additional role within the company.

NOTE

Waymo is a self-driving technology company spun out of Google's X incubator in 2016. It has since deployed the world's first fully autonomous ride-hailing service to the public in half a dozen major cities (with more to come). As a technology company that utilizes both complex distributed systems in the cloud and cutting-edge technology in the vehicle itself, Waymo is a perfect example of how technology companies can bridge the gap between the digital and physical worlds. This is a useful showcase for how the SRE role can apply principles of reliability, resilience, and safety in nontraditional ways.

Traditional role of SRE on-call

At many organizations, there are multiple on-call echelons that an SRE might participate in. Every SRE will be on-call for their local team or product (their core rotation). Some SREs may have enough context to jump into any incident within their larger organization. At Google, there is a higher level of on-call rotation: senior SREs with a particular penchant for IMAG have the option to join the engineering-wide on-call rotation called the Tech Incident Response Team (Tech IRT). This is a central team that helps coordinate, mitigate, and/or resolve major service outages for any product at the company.

At Waymo, the upper-echelon on-call rotation that an SRE can join is similar to Google's Tech IRT, but with an even broader scope: Waymo IRT is not focused specifically on "tech" at all. Waymo SREs handle outages for their local teams as a typical SRE team would. They also lead the Waymo Incident Management Plan, serving as both coordinators and scribes during major incidents in a role that extends beyond software incident response.

As described in Chapter 10, incident management in SRE traditionally focuses on troubleshooting, investigation, and mitigation. Waymo emergencies—particularly those that arise on the road or in vehicle depots—present a more physical observability and human information-gathering challenge. Reconciling system states with the real world while coordinating multiple crucial business functions, often in safety-critical situations, requires a higher-level incident management role that oversees not just the incident itself but also the *many independent mitigations that need to proceed simultaneously*. Waymo SRE has, in effect, leveraged reliability engineers to extend the Tech IRT model to encompass not only the broad technical mitigations but also the safety, legal, communications, and operational aspects required to fully resolve crises.

SRE coordinating a large-scale crisis

The bedrock of Waymo's culture is safety first. When the SRE organization was first established at Waymo, its system reliability roots from Google were merged with Waymo's hyperfocus on road safety. While Waymo must defend against server outages, it must also prepare for a wide array of operational and safety-related crises. To meet these business needs, Waymo SRE adopted a role unique among SRE teams at Alphabet. For a wide variety of safety-oriented events that are not purely engineering-focused (such as on-road incidents, physical security issues, and location-specific real-world events), Waymo SRE will activate the incident management plan and lead the cross-functional response.

In this company-wide incident response model, SRE is at the helm but relies on on-callers from legal, safety, ops, communications, policy, product, and any other groups whose involvement might be needed for the specific event. Note that these are not engineering teams, and the types of events escalated in this way range from partially engineering-related to not at all. In these cases, Waymo SRE assumes the role of incident coordinator (described momentarily) and is not responsible for actual mitigations. This is the primary way that Waymo's IRT differs from traditional SRE on-call rotations during an emergency.

The core tenet of structured incident response is the establishment of clear roles and expectations, allowing responders to dedicate cognitive power to mitigation rather than figuring out who to call or what they should be doing. In this model, SRE leads and facilitates incident response and resolution through two independent, always-on roles (i.e., not a primary and secondary, but two primary roles that are activated simultaneously):

- The *incident coordinator* is the overall incident and meeting facilitator. They do not mitigate issues on their own, but they know all of the subteams' roles and when to activate their playbook. The coordinator keeps the group on track and processes the event at a high level, splitting out breakout groups if needed to hammer out details without disrupting the main group. They manage dissenting opinions and ensure that safety and mitigation remain the priorities. They are the *conductor* of the IRT's symphony: they do not play an instrument themselves, but they direct the musicians.
- The *incident scribe* is the primary recordkeeper and screen presenter during the meeting. They make sure that the current state is well understood and the progression of the event is documented. The scribe calls out potential misalignments ("You just said *foo,* but the latest update from the field was *bar.* Which is correct?") and requests formal updates when key information is missing from the timeline. This role is especially critical in prolonged events, where accurate information is key to ramping up people who join late and for retracing steps during the inevitable retrospective writeup. Note that despite the name, this role *should not be* just a stenography task. When scribes are doing their jobs well, they are vital partners: multimedia presenters, fact-checkers, and gut-checkers, as well as note takers. They work in tandem with coordinators to keep the incident meeting flowing and effective.

Together, the coordinator and scribe guide the incident response while consulting the assembled on-call experts. They utilize predefined decision matrices and escalation policies, bringing in additional or higher-level stakeholders as needed. Once the incident has been resolved (or at least mitigated), the pair can call an end to the response effort. All that remains are the traditional post-incident tasks, delegated to appropriate owners: cleaning up any temporary incident measures, writing up a retrospective, and implementing action items to prevent similar issues in the future.

Why SRE can manage a crisis and not just an outage

Waymo SRE also does traditional SRE incident management for distributed systems and customer-facing services, but what differentiates the team are these additional, broader incident management responsibilities. Waymo SRE's coordinator and scribe have non-engineering responsibilities. Their roles are analogous to the incident commander and communications lead roles from traditional IMAG, but the incidents they handle at Waymo

are not limited to software engineering; they may range from injury and property damage to regulatory response or more. So why have reliability *engineers* taken ownership of these roles to solve *non-engineering* problems? There are a number of reasons:

- SREs are a mix of incident responders and engineers. While the coordinator and scribe roles do not require engineering expertise, they are the epitome of pure incident management. Other roles at Waymo may know more about the details of a given event, but none are as continuously trained in incident management. Letting SREs serve as high-level facilitators reuses this unique skillset.
- The role of a coordinator or incident commander is always to ensure the issue is mitigated—*not to resolve it themselves* (see Chapter 10). This positions SREs as experts in the *incident management process* itself, not necessarily as experts in the system involved in the incident.
- SREs *are also* engineers, and at Waymo the SRE team is on-call for the tooling used to escalate and organize an incident-level event. The same is true for several of the key tools frequently used during mitigation. As a result, while the core events are typically not within SRE's domain of expertise, much of the critical supporting infrastructure and process tooling is.
- On occasion, large company-wide events *are* distributed system issues—in which case one team is trained to handle, communicate, and resolve both the technical and operational aspects seamlessly.
- As discussed throughout this book, SREs focus on process improvement and automation. Incident retrospectives often contain tooling or policy action items that are best owned by SRE. Being tightly integrated with the major event handling process allows SREs to improve organizational processes and tooling through firsthand experience.
- By training an existing SRE team to lead company-wide, nontechnical IRT rotations, the company gains structure and coordination. This leads to shorter outages and reduced severity of issues, without duplicating work or requiring significant additional headcount.

Ultimately, the idea of using incident management for safety-forward, non-engineering emergencies is not new—organizations and governments have been doing so in some form for decades. The novel aspect of Waymo's approach was reusing an existing specialist engineering role—SRE—to apply its outage management skill set to broader incident management across the company.

When SRE is—and is not—a good fit for crisis coordinator

SREs owning broad incident management practices does not fit every business or industry. There are, however, simple hallmarks to consider when evaluating this expanded

role for SREs: the optimal "Goldilocks" organizational size and the diversity of job functions. If your company meets these criteria, it may achieve better use of human resources and more effective incident management outcomes by opting for the SRE-led incident model.

The first factor to consider is a company's relative size. Exact sizes will clearly vary, but in general this model is not a good fit for very small companies (such as early startups) or very large companies (5,000+ employees). On the smaller end of the spectrum, most startups will operate in a more ad hoc manner during major emergencies, and it is likely that anyone who can help will be involved in "all hands on deck" events. On the other end, large enough companies will probably have the resources to staff dedicated full-time corporate incident responders, reducing the need for SREs to fill this role. The sweet spot for SRE-led management of major company incident-level events is after "everyone senior hop on the video call" becomes untenable (and it *does* become untenable, sooner than you might expect) and before "we have dedicated teams just to handle these emergencies."

The second important aspect is the diversity of job roles within the company. SRE managing nontechnical IRTs works best if the nature of the company and the anticipated events require broad cross-functional participation. This is because SREs serving as non-domain-expert incident managers provide the most value when many disparate roles need to work in concert, and SRE can provide incident management value by *coordinating* those efforts. If a company is almost entirely made up of engineers and expects primarily engineering events, a more traditional technical IRT role for SRE is sufficient. On the other hand, if marketing represents 75% of the workforce and SRE is only a small part of operations, it may make more sense for marketing leaders with greater context to serve as IRT leads. The guideline here is that if an SRE-led IRT would continually defer to a single group for most incidents, this model may not be a good fit.

SRE for Security and Privacy

SRE's incident management skill set isn't the only versatile asset that can be used cross-functionally. Successful reliability engineering depends on attitudes of constructive pessimism—that is, looking at a system and understanding where its weak points are, under what conditions and in what ways those weak points might fail, and how they might be reinforced, redesigned, or even removed.[5]

SRE is just one discipline that uses these lenses to manage risk within tolerable levels; several other roles, with whom SREs often interact, also share these attitudes—it's just

that the adversary varies. SRE's adversary is, in some sense, the simple fact that with sufficient scale, even one-in-a-billion chances happen frequently enough to need to be reckoned with. Adjacent roles consider other adversaries: most familiarly, in the security context, someone trying to abuse a system and use it for unintended purposes. Similarly, in the privacy context, an adversary might be attempting to use the system in unanticipated ways to discover information about other users that they shouldn't be able to know.

Modeling threats to privacy, reliability, and security is a key part of how an effective engineer assesses systems, both as designed and as instantiated.[6] While an SRE might identify insufficient capacity as a threat and consider load shedding, diverting traffic to other locations, or degrading the quality of responses as possible mitigations, a privacy engineer might appraise the same system for risk of reidentification and mitigate that with anonymization or strict limits on data access and retention.

Privacy, reliability, and security also share many day-to-day tasks, albeit flavored by their respective domains. Each function:

- Responds to incidents[7]
- Tries to learn from incidents and identify mitigations, preventions, or even architectural and structural approaches to make entire classes of failure impossible
- Builds consensus and ideally implements these responses
- Documents what's been learned for future benefit

When a new vulnerability is discovered, security and SRE teams often work hand in hand to patch it across the fleet as fast as possible, without compromising availability. The following sections describe some events that demonstrate the critical connection between security and reliability.

IT Disruptions After Crowdstrike Software Update

On July 19, 2024, businesses across the world reported widespread IT disruptions caused by the CrowdStrike Falcon security software (an endpoint protection platform providing a range of security services). The affected organizations spanned a wide range of industries. Google Cloud detected some customer Windows VMs running on Google Compute Engine (GCE) experiencing blue screens of death and crashloops. The VMs, which were running CrowdStrike Falcon, began to fail after a CrowdStrike software update. CrowdStrike quickly deployed a fix, but some customer impact remained.

As pointed out in Chapter 15 of *Building Secure and Reliable Systems*, "Most systems eventually fail. Your ability to investigate a complex system relies on a number of factors: in addition to having access to adequate logs and information sources for debugging, you need proper expertise." During the CrowdStrike event, Google's customer support teams received many requests. Their first task was to figure out who could determine contributing factors and fix the issue for customers, which required an understanding of roles and responsibilities. Even though GCE SRE seemed like the appropriate team to investigate, its scope didn't include incidents related to guest environment-specific issues; its focus was on the VM infrastructure. The second team support tried to reach was the Guest OS engineering team, who also couldn't help, as CrowdStrike is third-party software that customers install manually on their VMs and is not shipped as part of Google's Windows images. Ultimately, support reached out to CrowdStrike, and they collaborated to find a solution.

While Google Cloud services were not directly affected, the Google Cloud Team worked closely with CrowdStrike to help our customers resolve any remaining issues (for example, by running commands on a recovery machine in Google Cloud to remediate the affected files—see GitHub). Successfully responding to this reliability disruption required customer support teams to work across organizational boundaries, collaborating with both SRE and security teams to identify the root causes of the problem caused by the security software.

Log4j Vulnerability

Apache Log4j is an open source, Java-based logging framework that collects and manages information about system activity—an important capability for investigating issues. Log4j is free, simple to use, and effective in its intended function, making it popular among Java developers, who have embedded it into countless software packages.

In November 2021, a vulnerability was identified that could allow an attacker to compromise a system running Log4j and execute arbitrary code on the vulnerable server. The National Institute of Standards and Technology (NIST) published a critical CVE (Common Vulnerabilities and Exposures, a system for cataloging vulnerabilities) in the National Vulnerability Database with a severity of 10/10 (the highest level). According to the Cyber Safety Review Board, the vulnerability could be exploited by sending specially crafted log messages to Log4j. An attacker who could control log messages or their parameters could execute arbitrary code loaded from remote servers when message lookup substitution was enabled (see "Google Cloud Recommendations for Investigating and Responding to the Apache *Log4j 2* Vulnerability").

As the scope and urgency of the Log4j vulnerability came to light, Google's central security team engaged with Tech IRT to respond to the issue. Tech IRT activated teams within each department to create scalable response groups to address the issue across the organization. Each response group was required to execute a Code Red.[8] The exit criteria of the Code Red included multiple requirements, among which were upgrading all instances of Log4j to a safe version, filing exceptions where upgrading was temporarily not feasible, and documenting the status of each system a team was responsible for. These initial steps were coordinated centrally by SRE and security.

As the internal investigations progressed, communication and customer support teams worked on publishing security advisories, sending Mandatory Service Announcements (MSAs) when customer action was needed, and sharing relevant information.

The large-scale response required collaboration across SRE, security, technical program management, product, support, communications, and legal teams. One thing that went particularly well was defining an end-to-end process flow at the start of the Code Red. Additionally, maintaining an overview of products that had identified a risk and required remediation enabled centralized tracking of progress and code changes without requiring direct communication with a given point of contact. Executive and senior leadership sponsorship, engagement, and prioritization were critical during the response, which took place during the holiday season code freeze. One area for improvement that was identified was the verification process: product teams could have used more specific instructions about how to demonstrate that they had upgraded their systems to a safe version.

Aside from the specifics of the Log4j vulnerability event, the broader reason we believe strong collaboration between SRE and security is important is detailed in Chapter 1 of *Building Secure and Reliable Systems*. Recovering quickly, investigating systems and logs, thorough crisis management following the IMAG protocol, and the organizational structure of Tech IRT all contribute to an effective response to large-scale security incidents.

There's much for practitioners in software quality, privacy, reliability, and security to learn from adjacent disciplines: don't be afraid to examine what your counterparts have learned over the years and see whether there are lessons of value you can bring to bear in your organization.

Conclusion

Implementing SRE principles outside of major tech companies requires a cultural shift rather than a simple tooling mandate. The primary challenge is adapting proven reliability principles and practices into a flexible toolkit tailored to your organization. To succeed, you'll need to select the practices with the greatest impact (such as defining SLOs and SLIs, reducing toil, and encouraging a blameless postmortem culture) that align with your company's unique scale, budget, business priorities, and regulatory constraints. You and your organization must do the work to define what SRE means for you. Your adoption of SRE may manifest as platform engineering, a specialized on-call rotation, or an enhanced DevOps function. Once that work is done, you need to ensure the new SRE team has the autonomy and management buy-in necessary to enable the engineering work required to elevate your standard of reliability.

1 As recommended in "On-Call Considerations".

2 Generic mitigations are actions that can reduce the duration or impact of a wide variety of production outages, rather than being tailored to a single, specific failure scenario.

3 E.g., r9y.

4 As described in "Kitchen Sink".

5 Todd Underwood's talk "PostOps: A Non-Surgical Tale of Software, Fragility, and Reliability" at the 27th Large Installation System Administration Conference (LISA '13); the slides are also available.

6 For an overview of threat modeling, largely in the security context, see Chapter 14 of Heather Adkins et al.'s *Building Secure and Reliable Systems* (O'Reilly, 2020) and Gayathri Mohan and Jim Gumbley's "2025 article "Threat Modeling Guide for Software Teams". In the privacy context, see Cara Bloom's PEPR '22 talk "Privacy Threat Modeling". In the reliability context, see Chapter 12.

7 See Lea Kissner's presentation "When Things Go Wrong" at the PEPR Conference (2020) for privacy incident response. For an overview of the (extensive) security response considerations, see Alex Nelson et al.'s "Incident Response Recommendations and Considerations for Cybersecurity Risk Management", NIST Special Publication 800-61 Revision 3, 2025; and Chapter 17 of *Building Secure and Reliable Systems* by Heather Adkins et al. (O'Reilly, 2020).

8 Google uses color codes to denote the severity of different types of emergencies and strategic initiatives. Code Red is the highest priority code, for problems that are already causing major damage

or will cause damage in the next 24 to 48 hours. Code Yellow is the second highest, for problems that may cause business issues, or outages, over the next quarter.

Previous chapter

< 21. How AI Supports SRE

Next chapter

23. Future of SRE >